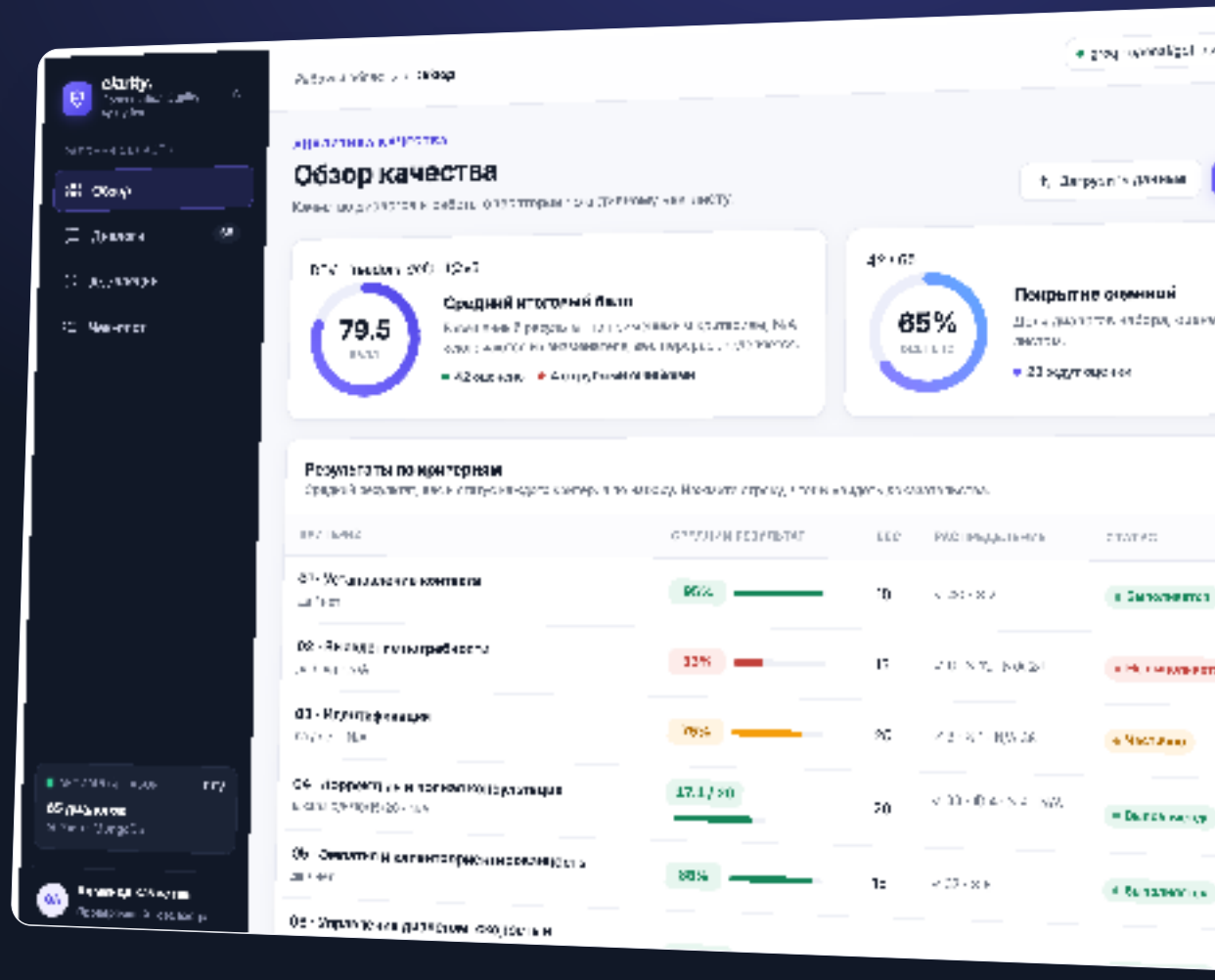
clarity.

FREEDOM BROKER · CQA HACKATHON · DATATHON FALL 2026

Conversation Quality Analytics

Automatic, explainable quality control of operator chats and calls — every score backed by the exact message, with appeals and a configurable checklist.

Team Kaira · RU · KZ · EN · Chats + calls



cqa-freedom.vercel.app · github.com/AkhmadAssentay/cqa-freedom

THE CHALLENGE

Quality control of operators does not scale by hand

A reviewer reads or listens to every conversation against a checklist. It is slow, only a small sample gets checked, and a score without proof is hard to explain or dispute.

1

Coverage

Manual review covers only a sample of chats and calls. Problems in the rest go unnoticed.

2

Explainability

An operator needs to see **why** points were lost and in which message — otherwise a score cannot be trusted or appealed.

3

Real-world data

Three languages (RU, KZ, EN), chats with bots and system events, and voice calls with holds and transfers.

What the brief asked for

- Evaluate a dialog by a **checklist**: answer, points, weight
- An **explanation** and the **evidence message** for every criterion
- **N/A** with a reason; **gross errors** flagged separately
- **Appeal** → reviewer decision → **recalculated total**
- **Configurable** checklist: criteria, weights, answer types, scales
- UI embeddable into **Dynamics 365**

OUR SOLUTION

Clarity: from raw dialog to an explained, disputable score

One web app for the whole quality loop. It runs online (Vercel + MongoDB Atlas + Groq) or fully locally with a free local model.

1 · LOAD

Datasets

Fetch by group_id from the organizers' API or upload JSON. Chats and calls, RU / KZ / EN.

→

2 · EVALUATE

AI + code checks

One LLM call per dialog for all criteria; crisp rules are verified in code.

→

3 · EXPLAIN

Evidence for every point

Score, sections, weights, explanation and clickable evidence messages.

→

4 · REVIEW

Appeals

Operator challenges a criterion; reviewer confirms or changes it; the total is recalculated.

65

DEV dialogs evaluated (60 chats + 5 calls)

94.9%

agreement of our submitted DEV results with reference labels

90.8%

best automatic method on a held-out test split (model router)

<1¢

per evaluation online; $0 with a local model

CORE PRINCIPLE

The model points to messages — code checks every reference and computes every number

NORMALIZE

Transcript & facts

Speakers, timestamps, handover to operator, first-response time, hold periods in calls.

CODE

→

DETECT

Crisp rules

Greeting in the client's language, operator name, goodbye, client left early, system-verified identity.

CODE

→

JUDGE

LLM verdicts

Prompt built from the checklist; strict JSON slot per criterion; yes / no / N/A / scale step + quotes.

LLM

→

VERIFY

Evidence check

Every quote is matched to the transcript; a "yes" about the operator must cite an operator message.

CODE

→

SCORE

Deterministic math

Weights, scale steps, N/A redistribution, rounding, gross-error flag. Unit-tested.

CODE

Why it matters

The LLM never does arithmetic and never invents evidence: a wrong message number is corrected, an unfound quote is marked unverified.

Any checklist

The prompt is generated from the checklist config, so a new or edited criterion works without code changes.

Bots & system events

Bot and system messages are context only; the operator is judged from the moment of handover.

CHECKLIST & SCORING

Scores follow the checklist exactly — and are reproducible

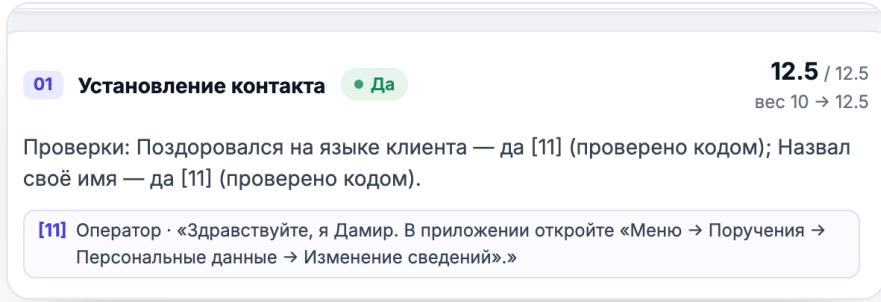
#	CRITERION	ANSWER TYPE	WEIGHT	N/A
01	**Contact establishment**	Yes / No	**10**	
02	**Needs discovery**	Yes / No	**15**	allowed
03	**Identification**	Yes / No	**20**	allowed
04	**Correct & complete consultation**	Scale 0/5/10/15/20	**20**	allowed
05	**Empathy & customer focus**	Yes / No	**15**	
06	**Dialogue management, speed, literacy**	Scale 0/4/7/10	**15**	
07	**Closing**	Yes / No	**5**	allowed
08	**Gross errors**	Flag	—	

Default checklist from the brief. Every edit saves a new version; past evaluations stay linked to theirs.

- **Yes** = full weight, **No** = 0, a **scale step** is proportional (top step = full weight)
- **N/A** only where allowed; its weight is **redistributed**, so the maximum stays 100
- Total rounded to 2 decimals; sections use the same formula
- **Gross errors** are a separate flag without points

WORKED EXAMPLE · P1-RU-007

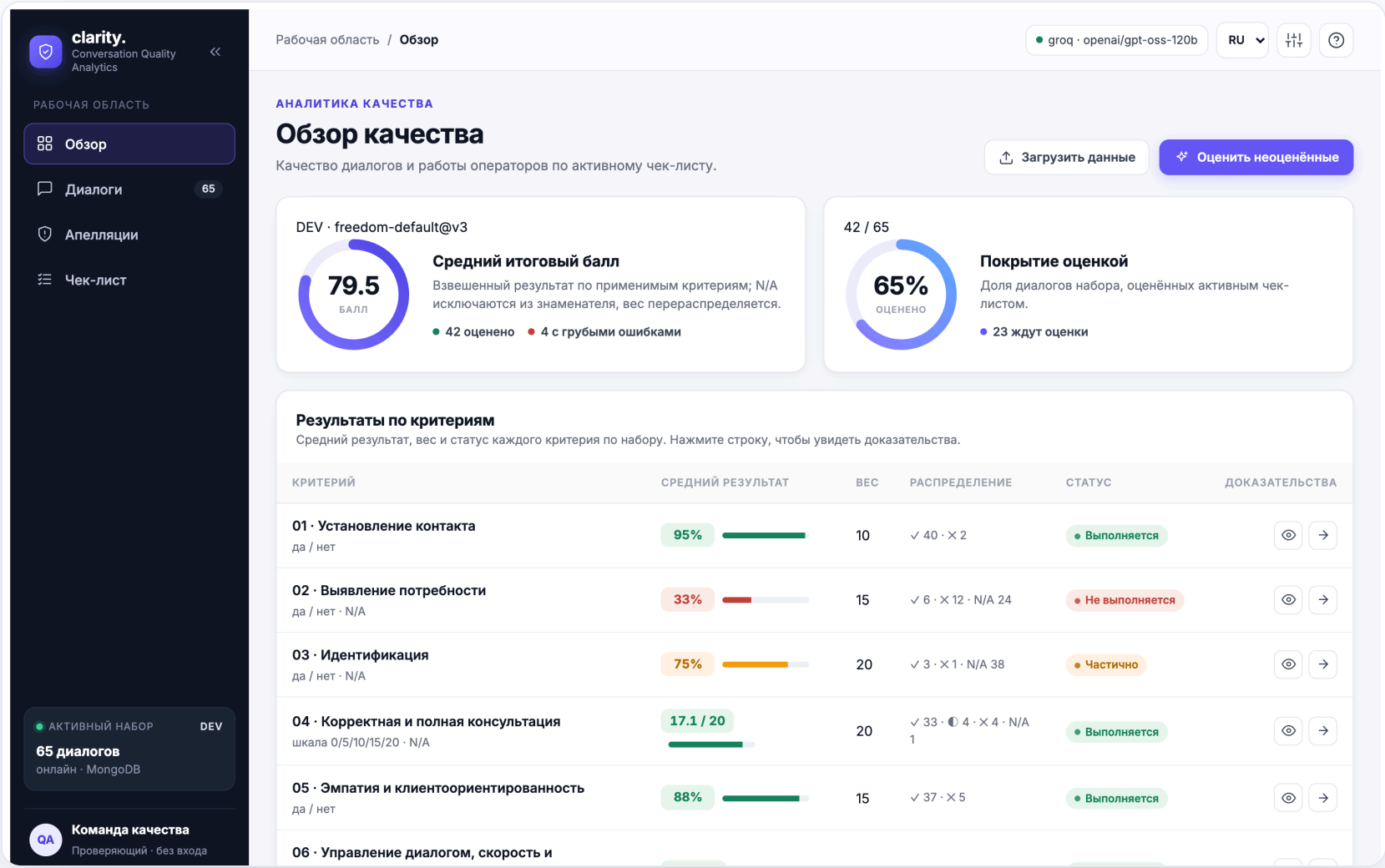
Identification is **N/A** (no personal data was shared), so its 20 points are spread over the other 80: Contact **10 → 12.5**, Needs discovery **15 → 18.75**, Consultation **20 → 25**.



PRODUCT · QUALITY OVERVIEW

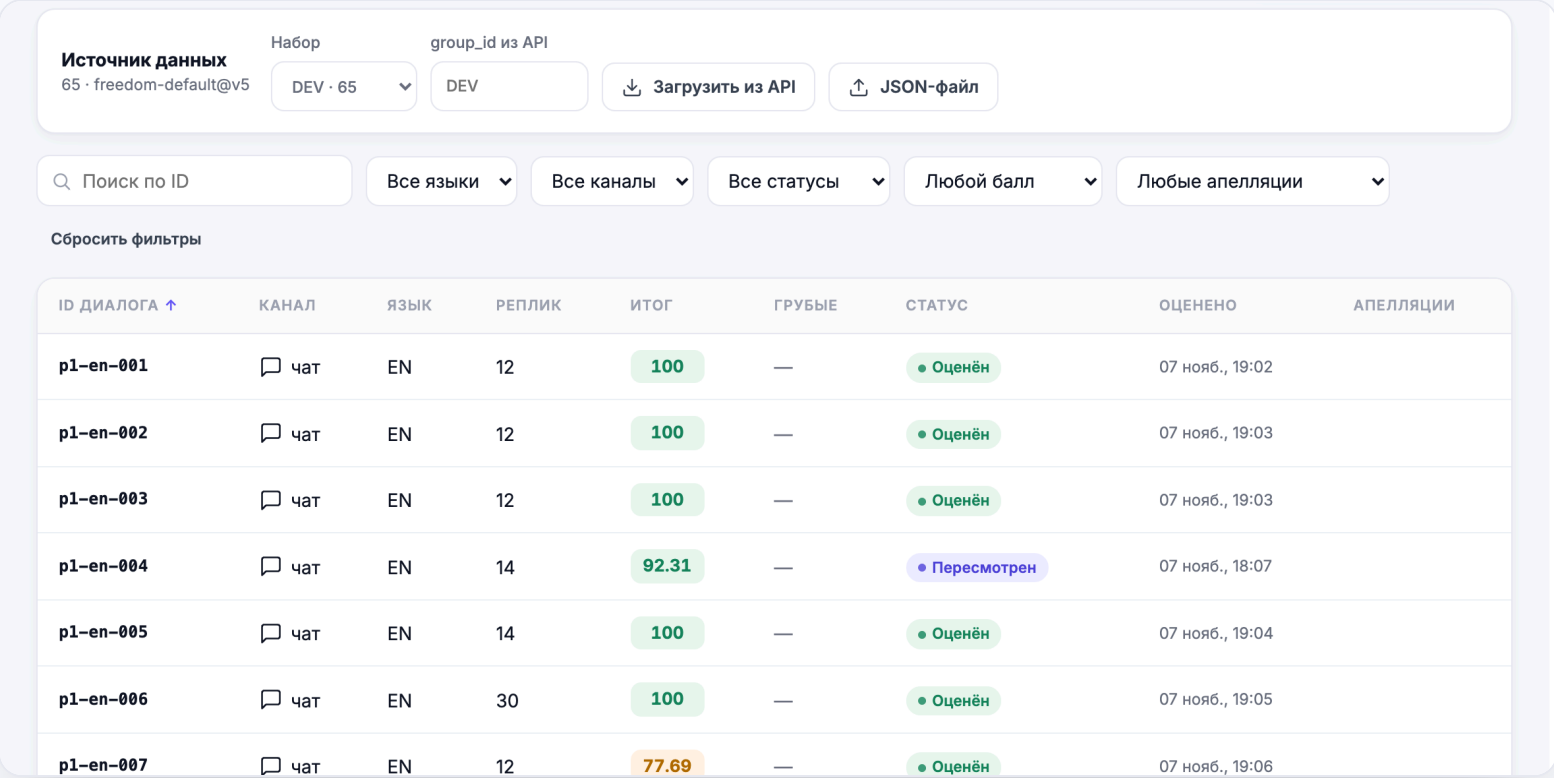
The whole dataset at a glance

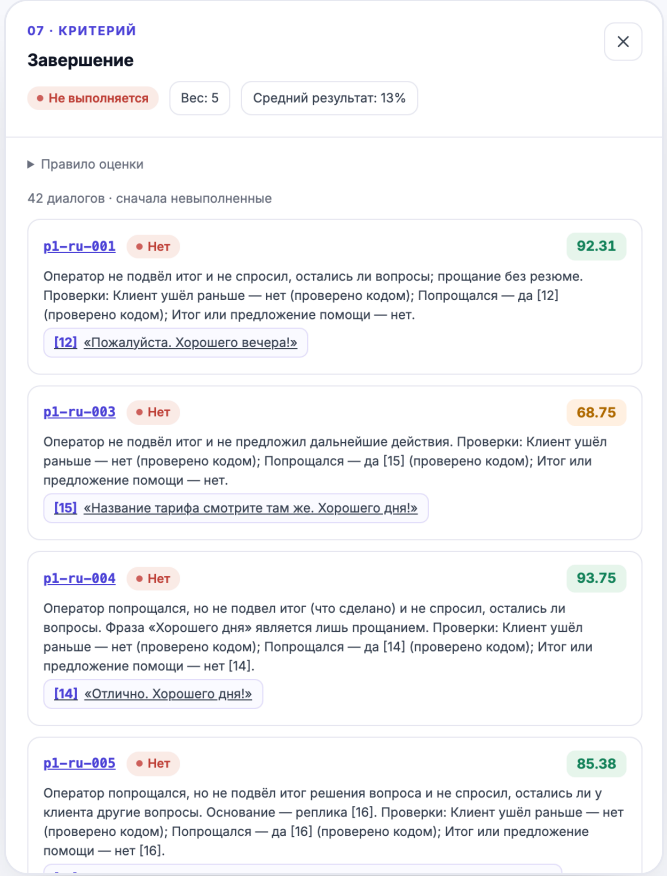
1. **Average score & coverage**
   Weighted final score across evaluated dialogs, and the share of the dataset already evaluated.
2. **Criteria performance**
   Average result, weight, distribution and status — Passed, Partial, Failed, Critical.
3. **One click to evidence**
   Each row opens every dialog's answer, explanation and quotes.
4. **Needs attention**
   Lowest scores, gross errors and open appeals.



PRODUCT · CONVERSATIONS & EVIDENCE

Find any dialog, see why a criterion passed or failed





Criterion side panel: every dialog's answer, explanation and quoted evidence, failed first.

Filters

Search, language, channel, status, score range, appeals.

Sort & pages

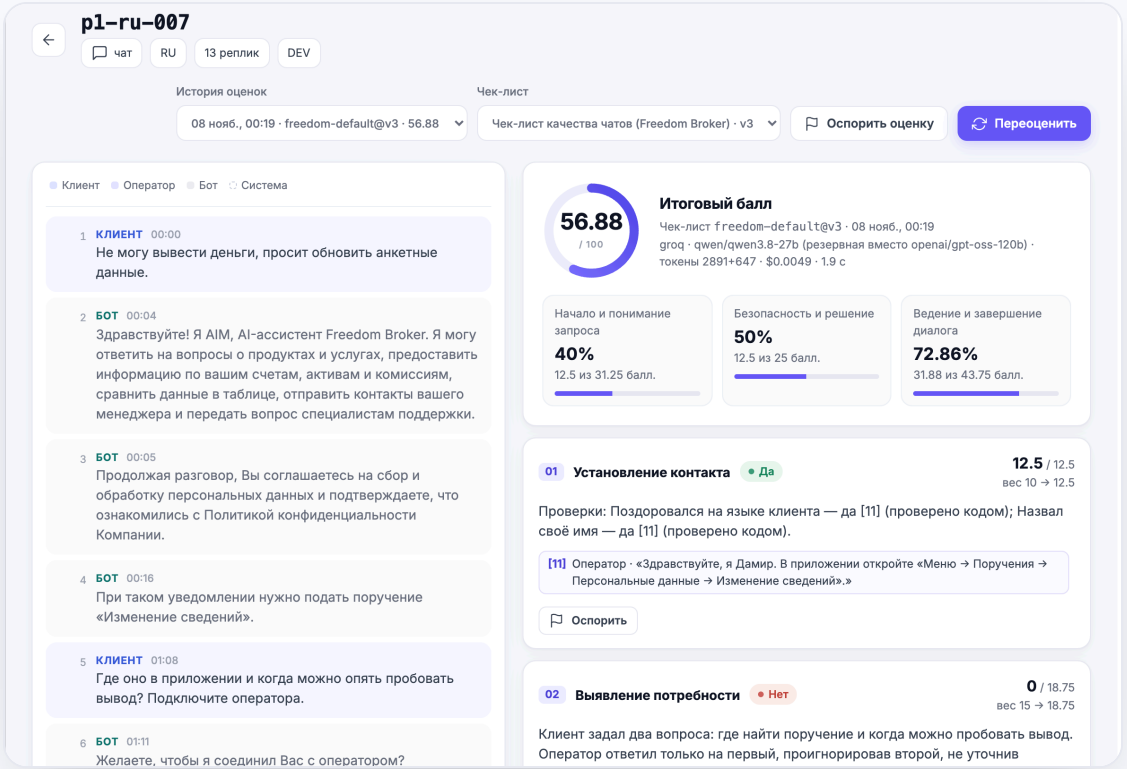
Sortable columns, 20 per page, evaluation date.

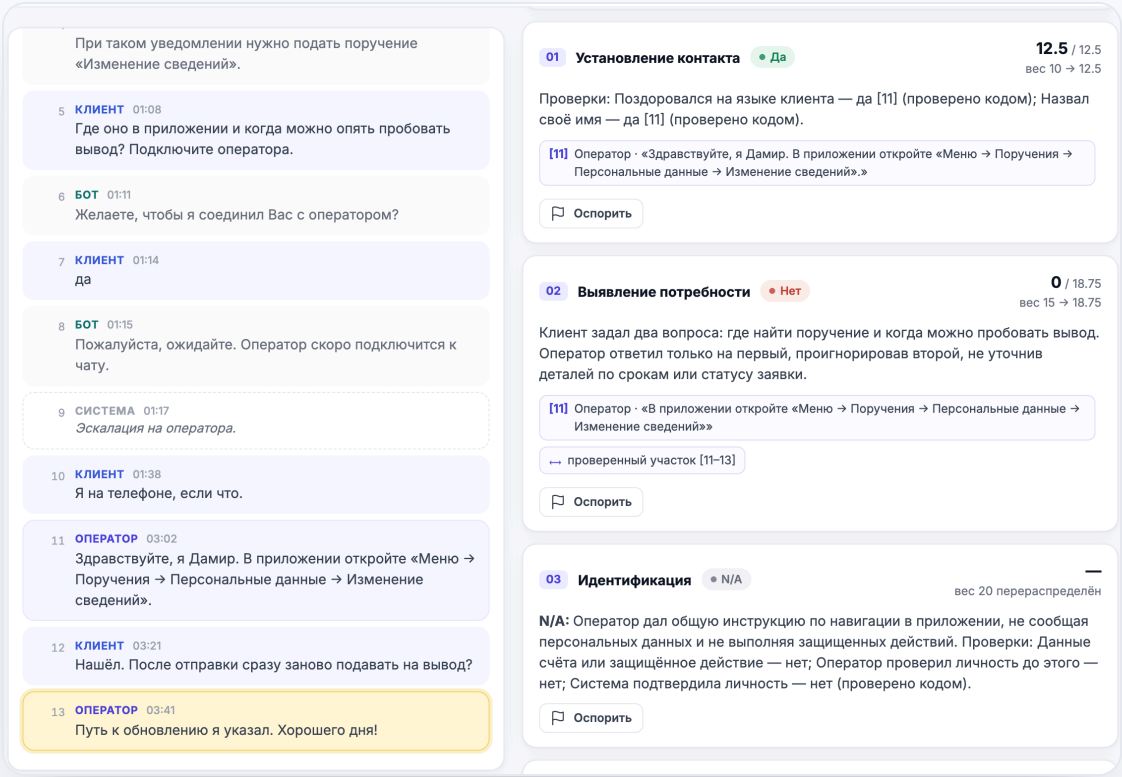
Batch

Evaluate pending or re-evaluate all with a progress bar.

PRODUCT · CONVERSATION CARD

Every point is linked to the message that justifies it





Speakers apart

Client, operator, bot and system are styled differently.

Click → highlight

Evidence scrolls to and highlights the message; for calls it seeks the audio.

Missed action

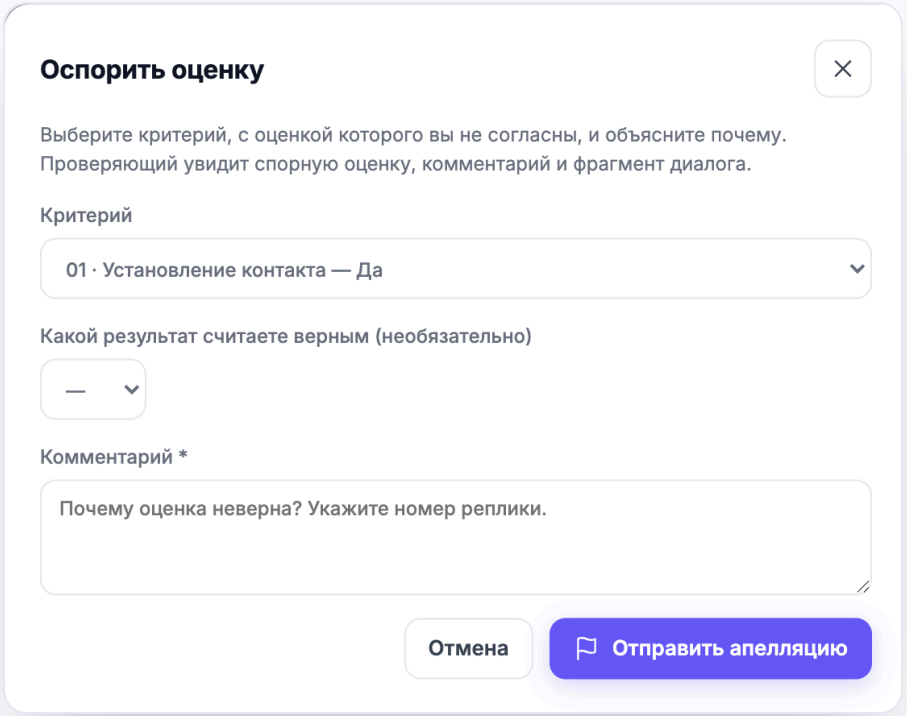
For "No", the checked range is shown, e.g. [11–13].

Transparent run

Model, tokens, cost, time and full evaluation history.

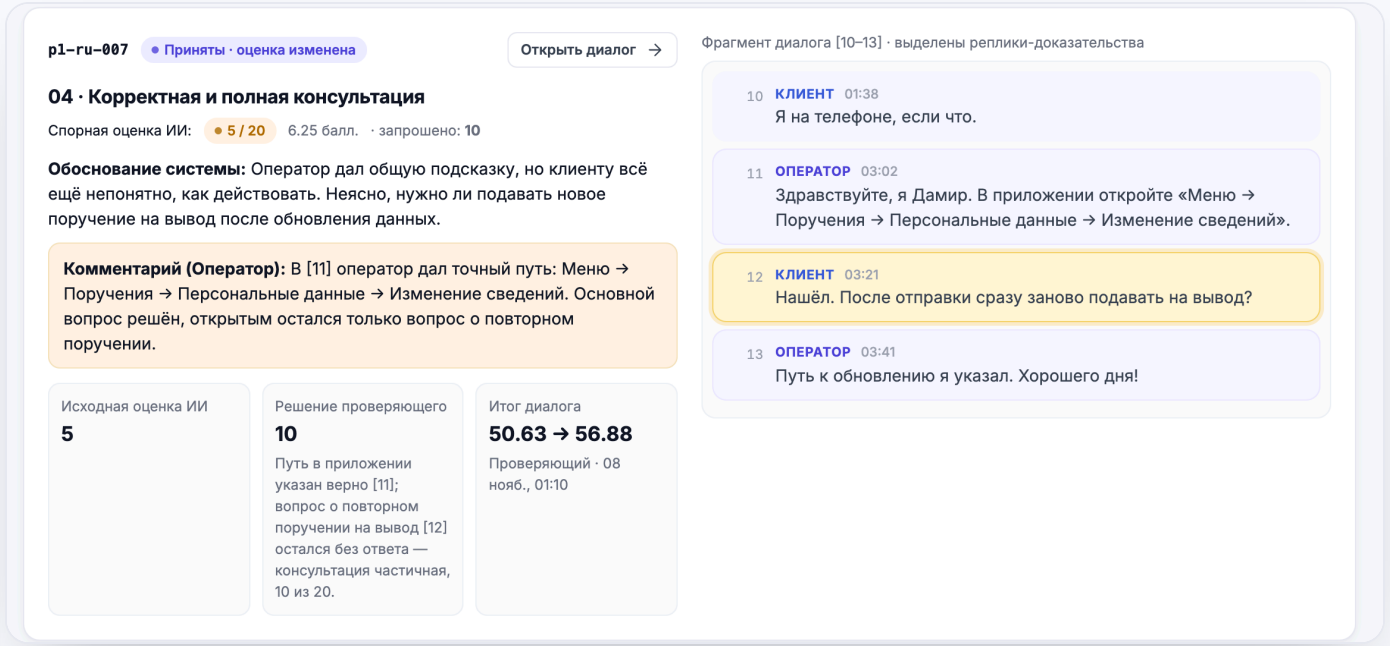
PRODUCT · APPEALS

Operator challenges, reviewer decides, the total updates itself



1 **Challenge evaluation**
Pick the criterion, optionally the answer you expect, and explain with message numbers.

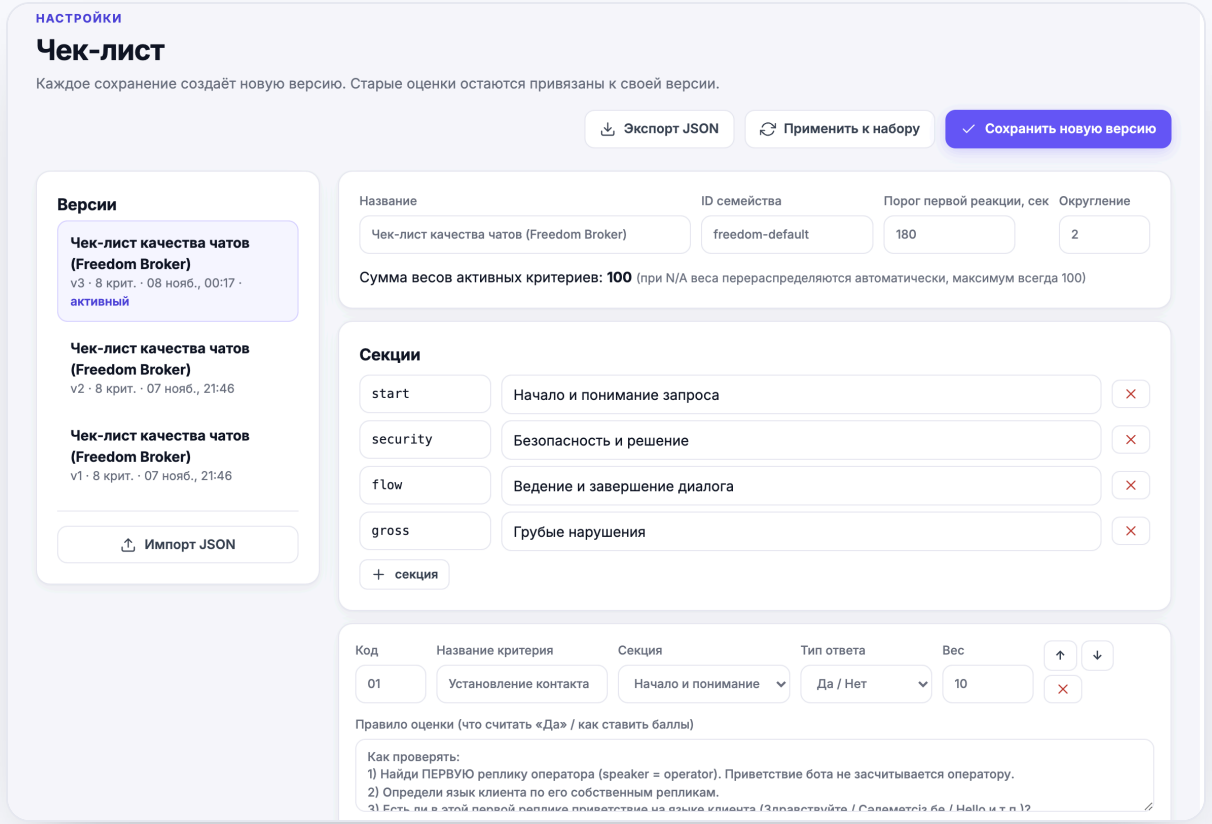
2 **Reviewer sees everything**
Disputed score, system rationale, comment and the dialog fragment with evidence highlighted.

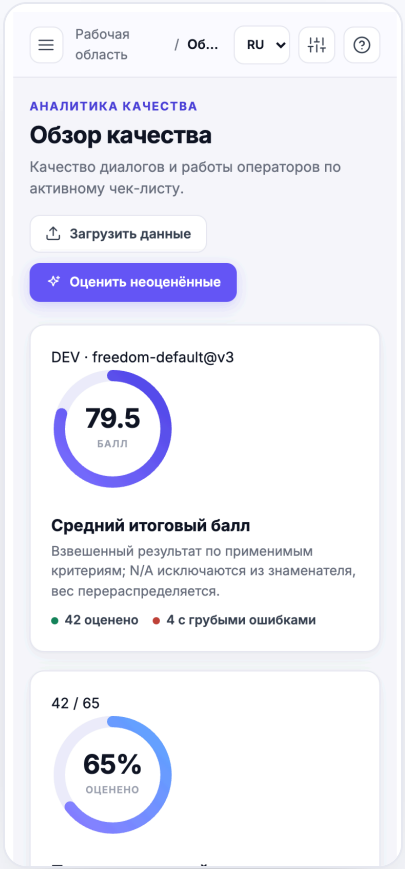


3 **Decision with a reason → recalculated total**
Original AI answer, reviewer decision and final score are all kept: here 5 → 10 points on consultation, total 50.63 → 56.88. Tabs: Pending · Accepted · Rejected · All resolved.

PRODUCT · CHECKLIST SETTINGS & ACCESS

QA leads change the rules without developers





- **Criteria:** name, rule text, section, weight, answer type (Yes/No, scale, flag)
- **N/A** allowed + condition; **scale steps**; reorder, add, delete
- **Active / inactive** switch — inactive criteria are skipped
- **Versions:** each save is a new version; import / export JSON
- **Apply to dataset** re-evaluates with the new version
- Interface in **RU / KZ / EN**, responsive down to phones
- Mounts into one element — ready for a **Dynamics 365** web resource

MAKING IT ACCURATE

Research-backed techniques we tested — kept only what measurably helped

✓

Code detectors

Greeting language, operator name, goodbye, early exit and system-verified identity are checked in code (RU/KZ/EN) — marked "checked by code" in the UI.

✓

Sub-questions + locked rules

For identification and closing, the model answers small yes/no questions; the verdict follows a fixed rule (CheckEval / TICK).

✓

Self-consistency

5 sampled answers from the local model, majority vote; low agreement marks the criterion "needs review" (Wang et al., 2023).

✓

Per-criterion routing

Each criterion goes to the model that is most accurate on it, chosen on the dev split and confirmed on test.

✓

Grounded evidence & facts

Quotes verified against the transcript; response times and pauses computed in code; on-hold internal talk in calls excluded.

×

Tried and dropped

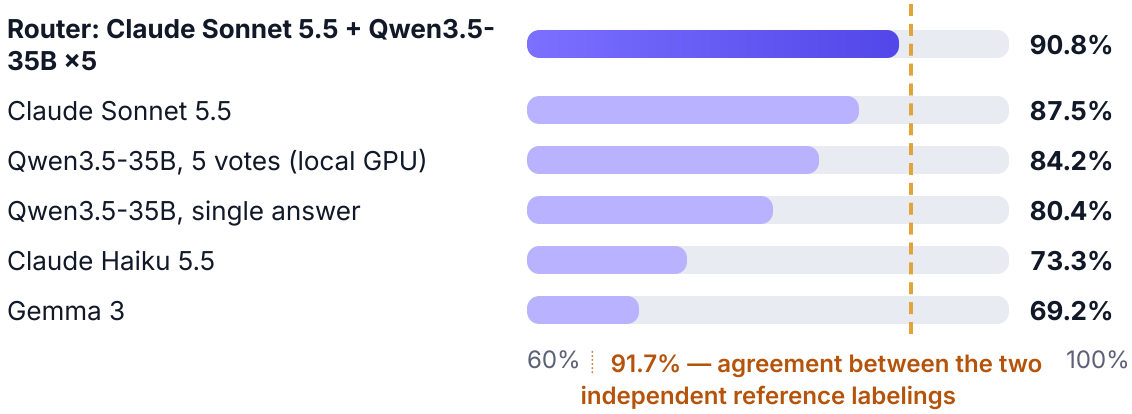
Appeal decisions as prompt precedents (90.8% → 87.9%), multi-model jury (PoLL), uncertainty cascades, KZ→RU gloss: all below the router.

MEASURED QUALITY

We measured every choice against reference labels

Method comparison · held-out test split

Exact-match accuracy over 8 criteria. Reference: 44 DEV dialogs labeled twice; the method is chosen on the dev split only.



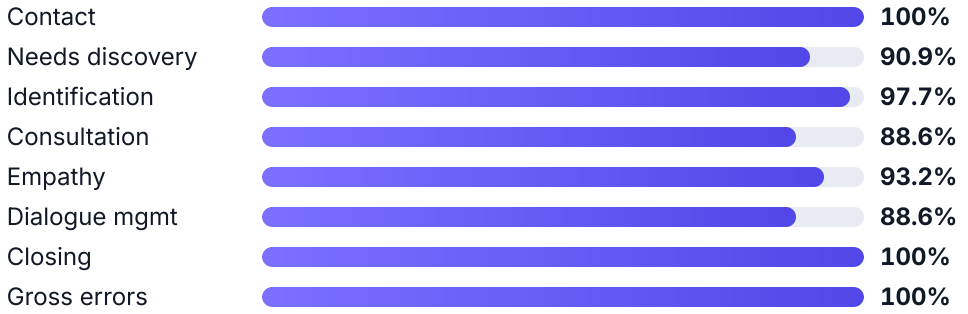
Router also caught 7 of 7 gross errors with no false alarms; final-score error 4.7 points.

Submitted DEV results

results/dev/results.json vs reference labels, 44 dialogs × 8 criteria

94.9% agreement

3.92 final-score MAE (points)



By language: EN 96.9% · KZ 96.9% · RU 94.9% · calls 79.2%

DEVELOPMENT PROCESS

Build, measure, fix — in short loops

1. **Understand the brief**
   Checklist modelled as configuration (JSON), scoring rules written as a spec and covered by unit tests first.

2. **Two prototypes in parallel**
   Python/FastAPI and Node.js versions explored different ideas; the best ones were merged (call holds, gross-error signals, a second labeling).

3. **Reference set & metrics**
   44 DEV dialogs labeled twice; harness for accuracy, Cohen's kappa, score MAE, false Yes / false No, N/A errors, cost.

4. **Model experiments**
   Claude Sonnet & Haiku, Qwen3.5-35B on a GPU via vLLM, Gemma 3, Qwen 2.5 7B on a laptop; jury, cascade and router compared.

5. **Reliability fixes from errors**
   Code detectors, evidence validation, first-response timing from handover, 1-based message numbers, appeals that really change the score.

6. **Go online & polish**
   Vercel + MongoDB Atlas + Groq with model fallbacks; results in Format JSON 1.0; Clarity UI redesign tested on every flow.

How we worked

- Every accuracy claim measured on held-out data, not on examples we tuned on
- Error analysis → targeted fix → re-measure (e.g. bot time no longer counted as operator delay: 7 dialogs corrected)
- Automated tests for scoring, rules and the LLM layer; every UI flow checked in a real browser
- Small commits on a shared GitHub repo; Vercel redeploys on every push

Privacy by default

- API keys only in environment variables, never in the repo
- Real call recordings are never served publicly
- Organizer data excluded from git; a local model keeps data in-house

ARCHITECTURE & DEPLOYMENT

Simple stack, no build step, runs online or fully offline

INTERFACE

Clarity UI

- Plain HTML / CSS / JS, mounts into one **#cqa-root** element
- Hosted as static files; embeddable as a Dynamics 365 web resource
- Read-only snapshot fallback if the API is down

SERVICE

Node.js API

- No npm dependencies; Vercel serverless function or **npm start**
- MongoDB Atlas online, JSON files locally — chosen automatically
- Evaluations are immutable; overrides and appeals stored apart

MODELS

Swappable LLMs

- Online: Groq **gpt-oss-120b → gpt-oss-20b → Qwen3** fallback on rate limits
- Local: Qwen 2.5 7B via Ollama, **$0**, no internet
- Anthropic, OpenAI or Gemini with one key

Research app (Python · FastAPI · SQLite)

Router of Claude Sonnet 5.5 and Qwen3.5-35B served with vLLM on a single RTX 5000 Ada GPU; hard budget cap for API spend; experiment harness and reference labels.

Cost

One evaluation ≈ 3–6K input tokens: about $0.002–0.005 on Groq, $0.015 for the Claude + Qwen router, $0 on a local model.

DELIVERABLES

Everything is live and reproducible

LIVE DEMO

cqa-freedom.vercel.app

Online mode: load a dataset, evaluate, appeal, edit the checklist — data in MongoDB Atlas, evaluation by Groq.

SOURCE CODE

github.com/AkhmadAssentay/cqa-freedom

README with run instructions, unit tests, experiment harness and reference labels.

RESULTS · FORMAT JSON 1.0

results/dev/results.json

65
records, all valid

80.02
average final score

8
dialogs with a critical flag

1-based evidence turns, rationale per criterion, validated by a schema checker.

CONTROL SET

results/prod/results.json

Prepared in the same format; filled as soon as the organizers publish the control dataset.

```
cd alt/node-prototype
cp .env.example .env  # one LLM key or Ollama
npm start  # → http://localhost:3000
```

HONEST LIMITS & NEXT STEPS

What we would do with more time and real data

Limitations today

- Reference set is small (44 dialogs) — figures are estimates
- **Calls** are the hardest part (79.2% agreement): long, mixed RU/KZ speech
- Intonation is not judged from audio; message-to-audio timing is approximate
- Free hosted-model quotas limit large online batches

Next steps

- **Dynamics 365:** swap the REST provider for Xrm.WebApi / Dataverse
- **PII masking** before any external API, or the local model only
- Grow the reference set with the QA team; recurring regression checks
- Word-level audio alignment (Whisper) for exact jumps in calls
- Operator-level trends and coaching reports from appeal outcomes

VALUE FOR FREEDOM BROKER

Every conversation checked

Not a sample — the whole flow of chats and calls, in three languages.

Fair and transparent

Each point lost is tied to a message, and every appeal leaves an audit trail.

Owned by the QA team

Rules, weights and versions change in the UI, without a release.

clarity.

Thank you

Questions & live demo

Demo	**cqa-freedom.vercel.app**
Code	**github.com/AkhmadAssentay/cqa-freedom**
Team	**Kaira**

